



UNIVERSITÉ
DE LORRAINE



Rapport de projet

Nuage de Mots d'Actualité

Projet universitaire

Master 1 MIAGE

Rendu le 22/05/2026

Membres du groupe :

EL AOUDI Rim

FUMERON-LECOMTE Baptiste

KACHLER Théo

SAHRAOUI DOUKKALI Mouad

Table des matières

1. Introduction	3
2. Conception	4
2.1 Structuration des documents	4
2.2 Indexation	5
2.3 Optimisation des requêtes et pipeline d'agrégation	6
3. Développement	9
3.1 Structure du code source	9
3.2 Fonctionnalités réalisées	9
Mode Administration	9
Mode Consultation.....	10
Nuage de mots SVG.....	11
3.3 Collecte des articles (parsing XML).....	11
3.4 Fonctionnalités partiellement assurées	11
4. Conclusions et perspectives	13
4.1 Retour critique sur MongoDB	13
4.2 Difficultés rencontrées	13
4.3 Pistes d'amélioration.....	13
Sources	15

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du projet du cours de Structuration de Documents. L'objectif était de développer une application web capable de collecter des titres d'articles d'actualité à partir de sitemaps Google News, de les stocker dans MongoDB, et de proposer une visualisation sous forme de nuage de mots au format SVG.

Le sujet nous a paru intéressant car il touche à plusieurs aspects vus en cours : le parsing XML (les sitemaps étant du XML avec des namespaces spécifiques), le stockage NoSQL orienté documents, l'agrégation de données avec le pipeline MongoDB, et la génération de contenu structuré (le SVG). Notre équipe de quatre personnes s'est réparti le travail en distinguant les parties backend (collecte, base de données) et frontend (interface Flask, templates).

L'application comporte deux parties. La première sert à gérer les sources et à lancer la collecte. La seconde permet de consulter les articles, filtrer les résultats et générer le nuage de mots.

2. Conception

2.1 Structuration des documents

Nous avons travaillé avec deux collections dans la base SD2026_projet :

- G_FKES_sources : chaque document représente un abonnement à un site d'actualité. On y stocke l'identifiant de la source (par ex. « lemonde »), l'URL du sitemap, et la fréquence de collecte en heures.
- G_FKES_articles : chaque document représente un article collecté. On y trouve le titre, l'URL originale, la date de publication, l'identifiant de la source, et un tableau *mots_cles* qui contient les mots significatifs du titre après nettoyage NLP.

Le choix d'intégrer les mots-clés directement dans le document article (plutôt que dans une collection séparée) est un compromis volontaire. En cours, on a vu le patron de référence étendue (Extended Reference) qui consiste à recopier certaines données pour éviter les jointures coûteuses. Ici, c'est un peu la même idée : en pré-calculant les mots-clés au moment de l'insertion, on évite de devoir re-traiter le titre à chaque requête d'agrégation. Ça duplique un peu l'information (le titre contient déjà les mots), mais le gain en performance est significatif pour la génération du nuage de mots.

On a aussi considéré le patron polymorphique vu en cours, mais nos deux entités (sources et articles) ont des structures trop différentes pour être dans la même collection. Séparer les sources dans leur propre collection était plus logique et permet de lister les abonnements sans toucher à la collection d'articles.

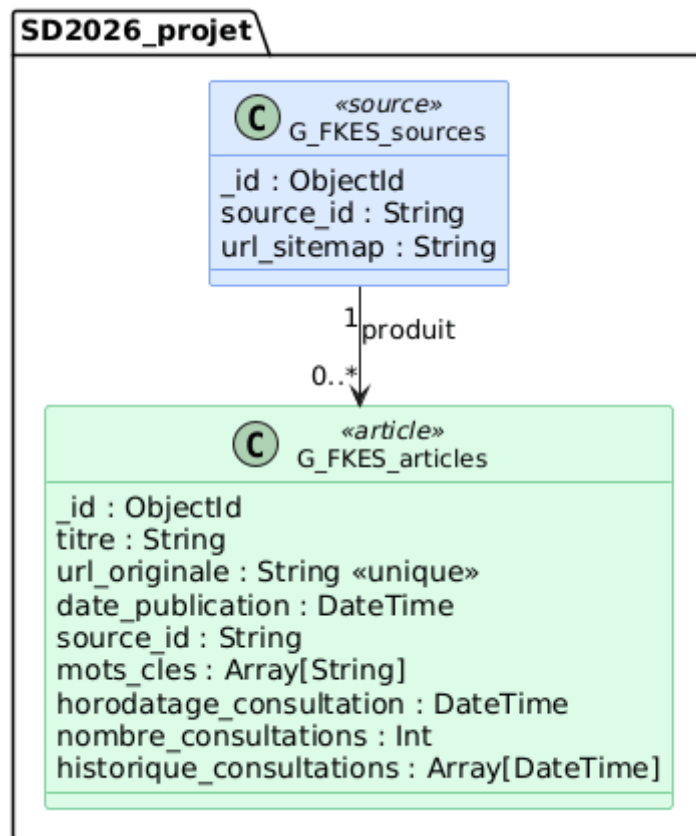


Figure 1 - Schéma de la base de données (collections et champs)

Concernant la gestion des consultations, quand un utilisateur clique sur un article, on stocke l'horodatage directement dans le document de l'article (champ `horodatage_consultation`). On maintient aussi un compteur `nombre_consultations` et un historique dans un tableau `historique_consultations`. Ce choix s'apparente au patron de sous-éléments (Subset) : les données de consultation sont embarquées dans l'article tant qu'elles restent de taille raisonnable.

2.2 Indexation

L'indexation a été un point important du projet. Comme vu en cours avec l'exemple de la collection `Inventory`, sans index, MongoDB fait un `COLLSCAN` (parcours séquentiel de tous les documents), ce qui devient lent dès que le nombre d'articles augmente. Nous avons mis en place les index suivants :

1. Index unique sur `url_originale` : Il garantit qu'on n'insère pas deux fois le même article. À chaque collecte, on utilise `update_one` avec `upsert=True` et `$setOnInsert`, ce qui est très efficace grâce à cet index.
2. Index descendant sur `date_publication` : les utilisateurs recherchent souvent les articles récents (derniers 7 jours, 30 jours...). L'index en ordre descendant accélère les requêtes avec `$gte`.
3. Index ascendant sur `source_id` : pour filtrer rapidement par source (« seulement Le Monde »).

4. Index composé (source_id + date_publication) : pour les requêtes qui combinent les deux filtres, ce qui est le cas typique quand un utilisateur sélectionne une source ET une période.

On a vérifié l'efficacité des index avec *explain("executionStats")* : sans index, une recherche sur la date parcourait tous les documents (COLLSCAN). Avec l'index, on passe à un IXSCAN qui n'examine que les documents pertinents.

2.3 Optimisation des requêtes et pipeline d'agrégation

Pour le nuage de mots, la question principale était : comment compter les mots les plus fréquents de manière efficace ? Deux approches étaient possibles :

- Côté Python : récupérer tous les articles, boucler sur les mots, compter avec un Counter. C'est simple mais ça transfère beaucoup de données du serveur MongoDB vers l'application.
- Côté MongoDB (pipeline d'agrégation) : déléguer le calcul au serveur de base de données. C'est ce qu'on a choisi.

Le pipeline utilisé est le suivant :

```
db.G_FKES_articles.aggregate([ { $match: { date_publication: { $gte: date_limite } } }, { $unwind: "$mots_cles" }, { $group: { _id: "$mots_cles", count: { $sum: 1 } } }, { $sort: { count: -1 } } ])
```

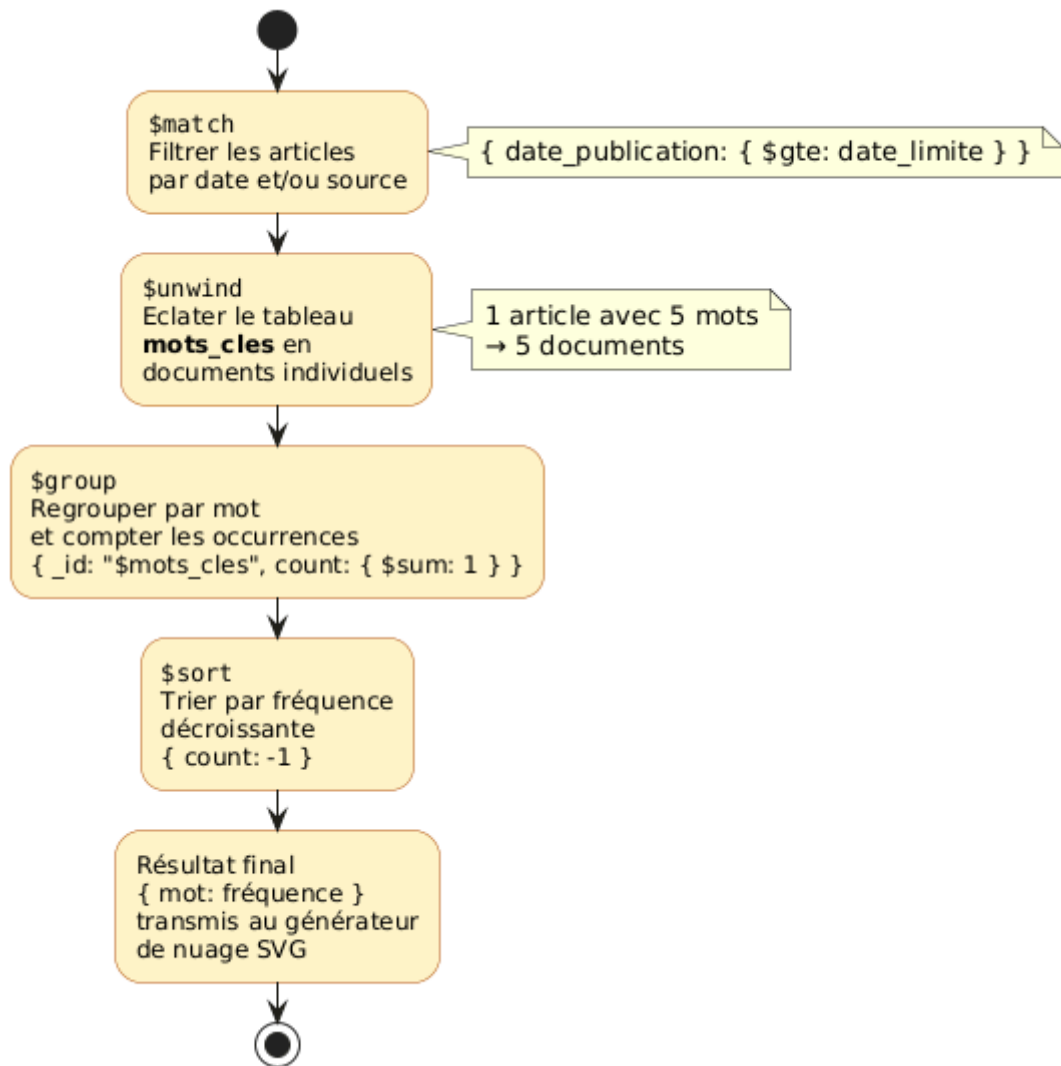


Figure 2 - Étapes du pipeline d'agrégation pour le nuage de mots

Nous avons repris le principe vu en cours avec l'agrégation sur Orders (agrégation \$match puis \$group), sauf qu'on ajoute un \$unwind pour éclater le tableau mots_cles en documents individuels avant de les regrouper. L'avantage, c'est que tout le traitement se fait côté serveur ; on ne récupère que le résultat final (une liste de mots avec leurs fréquences).

On a aussi implémenté une version Python « de secours » avec Counter (dans analytics.py, fonction *obtenir_frequence_mots*) au cas où l'agrégation poserait problème, mais en pratique c'est la version agrégation qu'on utilise.

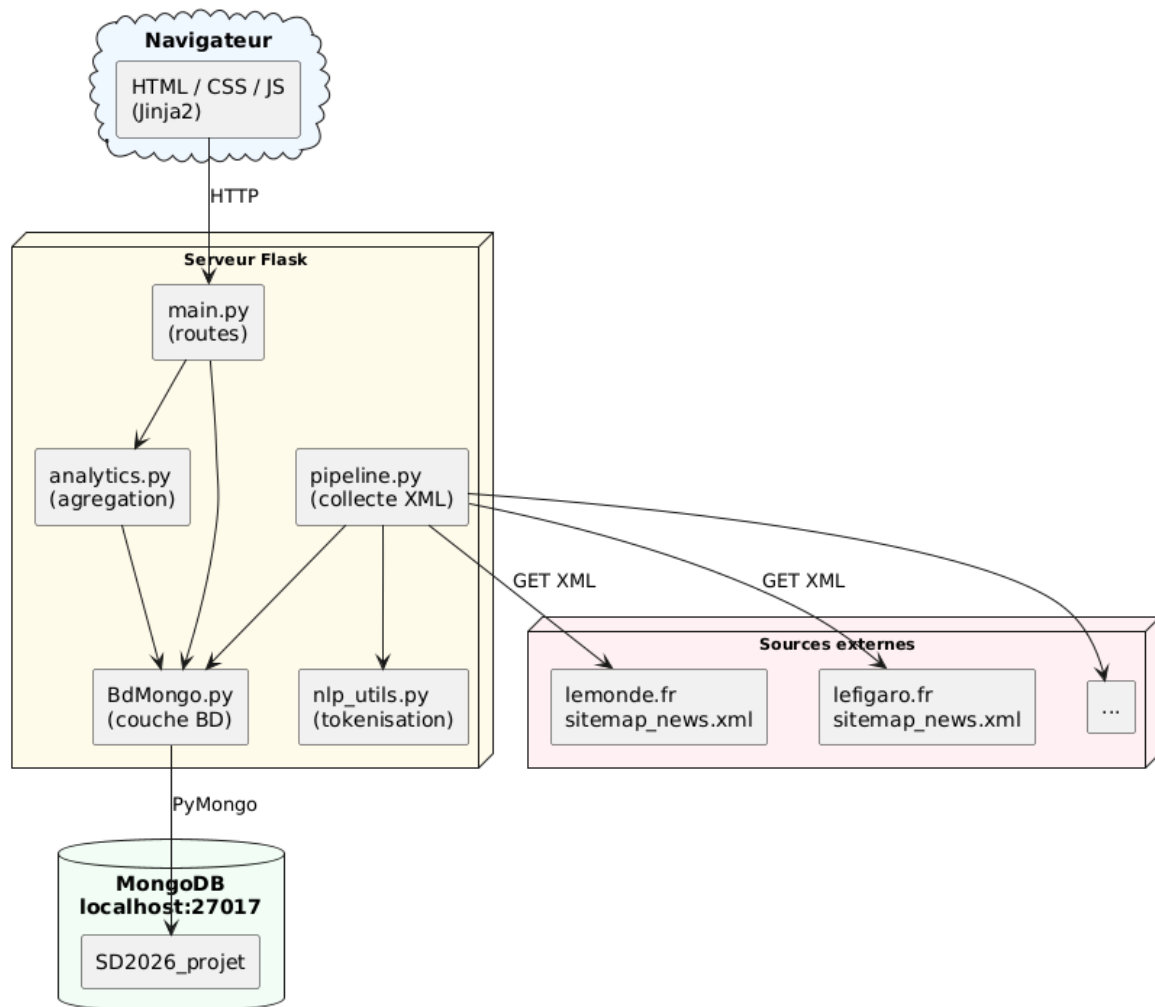


Figure 3 - Architecture générale de l'application

3. Développement

3.1 Structure du code source

L'application est organisée en plusieurs modules Python :

- `main.py` : point d'entrée Flask, définition des routes (`/`, `/articles`, `/admin`, `/api/wordcloud`, `/download/wordcloud.svg`, `/goto`).
- `BdMongo.py` : classe qui encapsule toutes les opérations MongoDB (connexion, CRUD articles, gestion des sources, création des index).
- `pipeline.py` : module de collecte qui parse les sitemaps XML et insère les articles. Utilise *xml.etree.ElementTree* pour parser le XML avec gestion des namespaces.
- `analytics.py` : module d'agrégation pour calculer les fréquences de mots (version pipeline MongoDB + version Python de secours).
- `nlp_utils.py` : nettoyage de texte : mise en minuscules, tokenisation par regex, suppression des stop-words français avec NLTK.

Côté frontend, on a quatre templates Jinja2 : `base.html` (layout commun avec navbar), `index.html` (page d'accueil avec nuage de mots), `articles.html` (consultation et recherche), et `admin.html` (gestion des sources).

3.2 Fonctionnalités réalisées

Mode Administration

L'interface d'administration permet d'ajouter une source (en saisissant un identifiant et l'URL du sitemap) et de la supprimer. Un bouton « Forcer la collecte » permet de déclencher manuellement le pipeline de scraping sans attendre le scheduler.

Le scheduler (`APScheduler`) est configuré pour lancer automatiquement la collecte toutes les 6 heures. On a utilisé `use_reloader=False` dans Flask pour éviter que le scheduler soit lancé en double en mode debug (c'est un piège classique de Flask).

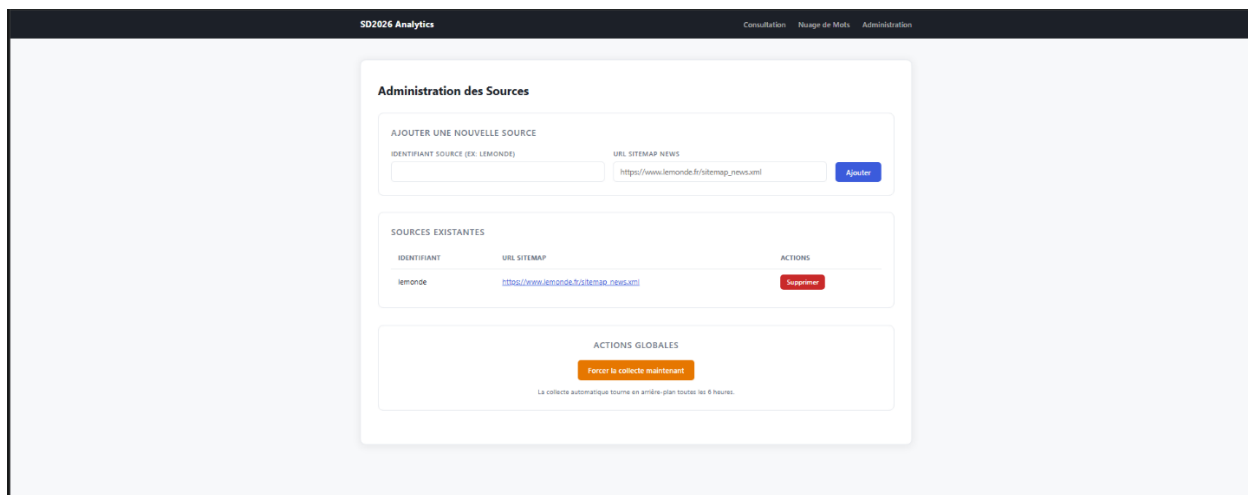


Figure 4 – Interface d'administration des sources

Mode Consultation

La page de consultation affiche les articles avec plusieurs options de filtrage : par source (via un menu déroulant), par période (nombre de jours), et par mot-clé (recherche regex insensible à la casse dans les mots-clés). Les résultats sont paginés (20 articles par page) et triés par date décroissante.

Chaque titre est cliquable et ouvre l'article dans un nouvel onglet (via `target="_blank"`). Le passage se fait par la route `/goto` qui enregistre l'horodatage de consultation en base avant de rediriger vers l'URL d'origine. On affiche aussi une section « Articles les plus consultés » basée sur le compteur de consultations.

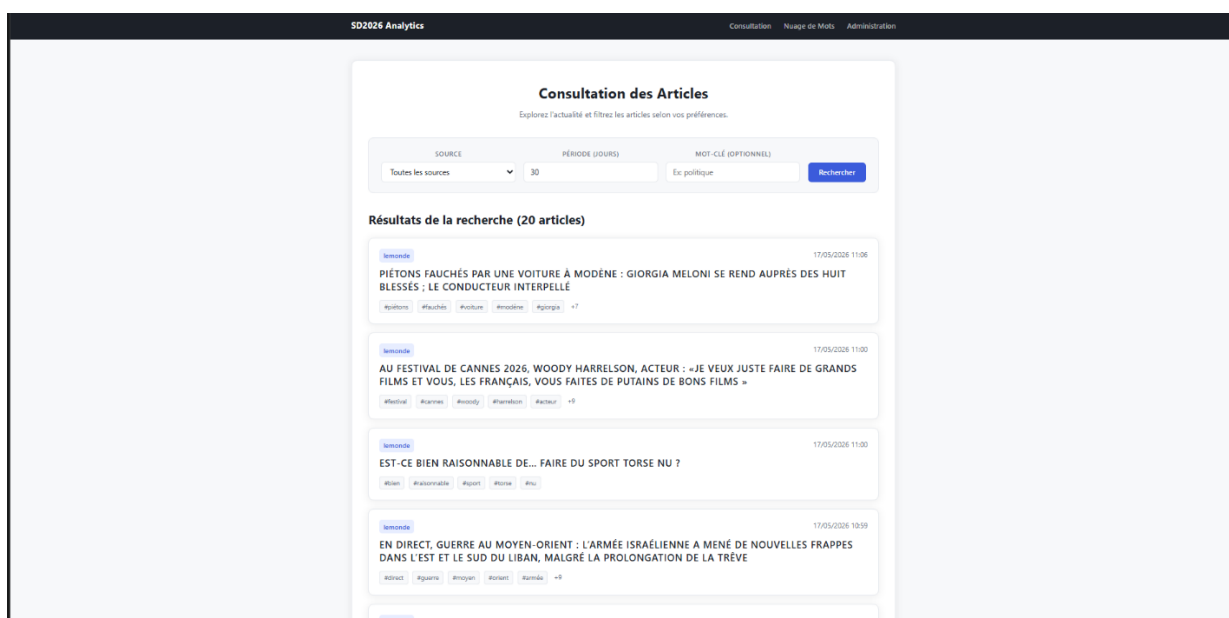
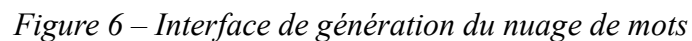


Figure 5 – Interface de consultation des articles

L'utilisateur choisit une source (ou toutes), une période, et un nombre maximum de mots. Le nuage est généré côté serveur avec la bibliothèque wordcloud de Python, qui prend en entrée un dictionnaire {mot: fréquence} et produit directement du SVG. On a rendu le SVG responsive en remplaçant les attributs width/height par un viewBox. Un bouton de téléchargement permet de récupérer le fichier SVG.



Le parsing des sitemaps Google News utilise `xml.etree.ElementTree`. Les sitemaps utilisent deux namespaces qu'il faut déclarer explicitement :

Pour chaque balise <url>, on récupère le <loc> (URL de l'article), le <news:title> (titre) et le <news:publication_date> (date). Le titre est ensuite passé dans notre pipeline NLP pour extraire les mots-clés significatifs.

La définition des minuteries **par source** n'est pas encore pleinement exposée dans l'interface : actuellement le scheduler utilise un intervalle global de 6 heures. Le champ *frequence_heures* existe bien dans la collection sources, mais on n'a pas eu le temps de créer un formulaire dédié dans la page admin pour le modifier par source. C'est une amélioration qu'on aurait pu faire avec plus de temps.

Le filtrage par « catégorie de mots-clés » mentionné dans le sujet n'a pas été implémenté non plus, car les sitemaps ne fournissent pas directement de catégorie. On aurait pu tenter une classification automatique des articles par thème, mais ça dépassait le périmètre du projet.

4. Conclusions et perspectives

4.1 Retour critique sur MongoDB

MongoDB s'est avéré adapté à notre cas d'utilisation. Les articles issus du scraping n'ont pas tous exactement la même structure (certains sitemaps peuvent avoir des champs supplémentaires), et la flexibilité du schéma NoSQL nous a permis de ne pas nous bloquer sur la définition d'un schéma rigide. Le pipeline d'agrégation est vraiment puissant pour notre besoin de comptage de mots : en une seule requête, on filtre, on éclate le tableau, on regroupe et on trie.

En revanche, l'absence de jointures natives est un inconvénient. Pour afficher le nom de la source à côté d'un article, on doit soit faire un \$lookup (qui est moins performant qu'un JOIN SQL), soit stocker le `source_id` directement dans l'article (ce qu'on a fait). C'est le genre de compromis qu'on n'a pas en relationnel.

Un autre point délicat : MongoDB ne vérifie pas la cohérence des données. Si on supprime une source, il faut penser à supprimer aussi ses articles côté applicatif, alors qu'en SQL un `ON DELETE CASCADE` le ferait automatiquement.

4.2 Difficultés rencontrées

Le parsing des dates dans les sitemaps a posé problème au début. Le format avec le fuseau horaire n'est pas toujours bien géré par `strptime`. On a fini par utiliser `datetime.fromisoformat()` après avoir retiré manuellement le décalage horaire.

Le scheduler `APScheduler` qui se lance deux fois quand Flask est en mode debug avec le reloader activé. On a résolu ça avec `use_reloader=False`.

La génération SVG du nuage : la bibliothèque `wordcloud` produit du SVG avec des dimensions fixes. On a dû modifier le SVG en post-traitement pour le rendre responsive avec un `viewBox`.

4.3 Pistes d'amélioration

Si on devait continuer le projet, voilà ce qu'on ferait :

1. Minuteries par source : ajouter dans l'interface admin un champ pour régler la fréquence de collecte individuellement pour chaque source, et adapter le scheduler en conséquence.
2. Index textuel : créer un index de type Text sur le champ titre pour permettre une recherche full-text plus performante qu'un regex.
3. Génération SVG native : au lieu d'utiliser la librairie `wordcloud`, générer nous-mêmes le SVG à partir des fréquences. Ça donnerait plus de contrôle sur le rendu : personnalisation des couleurs, polices, disposition, etc.
4. Collection séparée pour les consultations : si le nombre de consultations devenait très grand, le tableau `historique_consultations` pourrait alourdir les documents. On pourrait appliquer le patron d'aberration (Outlier) vu en cours pour gérer ces cas limites.

5. Ajout d'autres sources : tester avec des sitemaps d'autres journaux (Le Figaro, France Info, etc.) pour vérifier la robustesse du parser.

Sources

Bibliothèque NLTK Stopwords : <https://www.geeksforgeeks.org/nlp/removing-stop-words-nltk-python/>

Mongodb : <https://www.mongodb.com/docs/>

Apscheduler :

- Configuré le Scheduler: <https://apscheduler.readthedocs.io/en/3.x/userguide.html#configuring-the-scheduler>
- Lancer le Scheduler : <https://apscheduler.readthedocs.io/en/3.x/userguide.html#starting-the-scheduler>
- Ajouter des jobs : <https://apscheduler.readthedocs.io/en/3.x/userguide.html#adding-jobs>
- Eteindre le Scheduler : <https://apscheduler.readthedocs.io/en/3.x/userguide.html#shutting-down-the-scheduler>

Atexit pour la gestion de fin de programme : <https://docs.python.org/fr/3.9/library/atexit.html>

Wordcloud : <https://pypi.org/project/wordcloud/>

Pymongo : <https://pymongo.readthedocs.io/en/stable/>

Request : <https://requests.readthedocs.io/en/latest/>

Contenu du cours :

Structuration de documents fourni par Hendry FERREIRA CHAME.